

Tensorflow II

Python Image Libraries

Fatemeh Dehrouyeh



1 Python Image Libraries

- OpenCV
- Pillow
- scikit-image
- TensorFlow
- PyTorch

Current Section

1 Python Image Libraries

- OpenCV
- Pillow
- scikit-image
- TensorFlow
- PyTorch

Current Subsection

1 Python Image Libraries

- OpenCV
- Pillow
- scikit-image
- TensorFlow
- PyTorch

Starting with OpenCV

OpenCV is one of the most widely used libraries for image processing.

With OpenCV, we can:

- load images,
- inspect their dimensions and data type,
- access pixel values,
- convert color spaces,
- resize and crop,
- apply filters and transformations,
- save processed images.

Import

```
import cv2
```

Loading an Image with OpenCV

OpenCV loads an image using:

```
cv2.imread()
```

The result is stored as a NumPy array.

Important

If OpenCV cannot find or read the image, the returned value is None.

Code

```
▶ import cv2

image = cv2.imread("astronaut.png")

print(image)
print(type(image))

... [[ [151 147 154]
      [124 103 109]
      [102  58  63]
      ...
      [115 120 127]
      [106 117 120]
      [110 119 125]]

      [[171 171 177]
       [143 141 144]
       [124 114 113]
       ...
```

Inspecting an OpenCV Image

After loading an image, always inspect three things:

- shape
- dtype
- pixel value range

For a color image:

$$\text{shape} = (H, W, C)$$

where

H = height, W = width, C = channels.

Good Habit

Always inspect the data before processing it.

Code

```
▶ print("Shape:", image.shape)
  print("Data type:", image.dtype)
  print("Minimum value:", image.min())
  print("Maximum value:", image.max())
```

```
... Shape: (512, 512, 3)
     Data type: uint8
     Minimum value: 0
     Maximum value: 255
```

Important: OpenCV Uses BGR

Most image libraries represent color images as

RGB

but OpenCV loads color images by default as

BGR

The channel order is therefore:

Blue → Green → Red

instead of

Red → Green → Blue.

Common Mistake

Displaying an OpenCV image directly with Matplotlib may produce incorrect colors.

Compare BGR and RGB

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("astronaut.png")

plt.imshow(image)
plt.axis("off")
plt.show()
```

...



Then fix it:

```
▶ image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  
  
plt.imshow(image_rgb)  
plt.axis("off")  
plt.show()
```

...



Displaying Images

There are two common ways to display an OpenCV image:

- `cv2.imshow()` for a separate OpenCV window
- `plt.imshow()` for notebooks and visualization

For notebooks, Matplotlib is often more convenient.

Remember

Convert BGR to RGB before displaying an OpenCV image with Matplotlib.

Matplotlib display

```
import matplotlib.pyplot as plt

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6, 6))
plt.imshow(image_rgb)
plt.axis("off")
plt.show()
```



For a normal Python script:



```
cv2.imshow("Image", image)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

"Image" is the window title, image is the actual image being displayed, cv2.waitKey(0) waits until you press a key, and cv2.destroyAllWindows() closes the OpenCV window.

Accessing Individual Pixels

Because an OpenCV image is a NumPy array, we can access a pixel using

```
image[row, column]
```

or equivalently

```
image[y, x].
```

For a color image, the result contains three values:

```
[B, G, R]
```

Example

The pixel at row 100 and column 200 is accessed with

```
image[100,200]
```

Read a pixel

```
▶ pixel = image[100, 200]  
  
print(pixel)
```

```
... [17 57 81]
```

```
▶ # unpack it:  
  
b, g, r = image[100, 200]  
  
print("Blue:", b)  
print("Green:", g)  
print("Red:", r)
```

```
... Blue: 17  
    Green: 57  
    Red: 81
```

Changing Pixel Values

Pixel values can also be modified directly.

For a BGR image:

$[0, 0, 255]$

represents pure red.

Changing pixels directly helps us understand that an image is simply an editable numerical array.

Remember

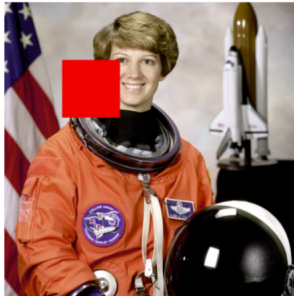
OpenCV uses the order **BGR**, not RGB.

Modify one pixel or region

```
image_copy = image.copy()  
image_copy[100, 200] = [0, 0, 255]
```

```
# A whole region is more visible:  
image_copy = image.copy()  
image_copy[100:200, 100:200] = [0, 0, 255]  
image_copy_rgb = cv2.cvtColor(image_copy, cv2.COLOR_BGR2RGB)  
plt.imshow(image_copy_rgb)  
plt.axis("off")  
plt.show()
```

...



Converting an Image to Grayscale

A color image contains three channels, while a grayscale image contains one intensity value per pixel.

OpenCV can convert BGR images to grayscale using

```
cv2.cvtColor()
```

The image shape changes from

$$(H, W, 3)$$

to

$$(H, W).$$

Grayscale

```
▶ gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Color shape:", image.shape)
print("Gray shape:", gray.shape)

plt.imshow(gray, cmap="gray")
plt.axis("off")
plt.show()
```

```
... Color shape: (512, 512, 3)
    Gray shape: (512, 512)
```



Resizing an Image

Neural networks often require images to have the same dimensions.
OpenCV resizes an image using

```
cv2.resize()
```

For example,

$$512 \times 512 \longrightarrow 224 \times 224$$

is common when preparing images for vision models.

Important

OpenCV specifies resize dimensions as

(width, height)

not (height, width).

Resize

```
▶ resized = cv2.resize(image, (224, 224))  
  
print("Original:", image.shape)  
print("Resized:", resized.shape)
```

```
... Original: (512, 512, 3)  
Resized: (224, 224, 3)
```

```
▶ # Display  
resized_rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)  
  
plt.imshow(resized_rgb)  
plt.axis("off")  
plt.show()
```



OpenCV expects resize dimensions in the order (width, height), while an image array is stored with shape (height, width, channels); the image itself is not reversed, only the dimension ordering convention is different.

Cropping an Image

Cropping selects only a region of an image.

Because OpenCV images are NumPy arrays, cropping uses array slicing:

```
image[y_1:y_2, x_1:x_2]
```

Cropping does not require a special OpenCV function.

Key Idea

Image operations often become simple NumPy array operations.

Crop

```
▶ crop = image[100:350, 150:400]

crop_rgb = cv2.cvtColor(crop, cv2.COLOR_BGR2RGB)

plt.imshow(crop_rgb)
plt.axis("off")
plt.show()
```

...



Flipping an Image

OpenCV can flip images horizontally or vertically using

```
cv2.flip()
```

Common flip codes:

- 1: horizontal flip
- 0: vertical flip
- -1: both directions

Flip

```
horizontal = cv2.flip(image, 1)
vertical = cv2.flip(image, 0)
both = cv2.flip(image, -1)
```

```
# Convert flipped images to RGB
horizontal_rgb = cv2.cvtColor(horizontal, cv2.COLOR_BGR2RGB)
vertical_rgb = cv2.cvtColor(vertical, cv2.COLOR_BGR2RGB)
both_rgb = cv2.cvtColor(both, cv2.COLOR_BGR2RGB)

# Show all images
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0, 0].imshow(image_rgb)
axes[0, 0].set_title("Original")

axes[0, 1].imshow(horizontal_rgb)
axes[0, 1].set_title("Horizontal Flip: cv2.flip(image, 1)")

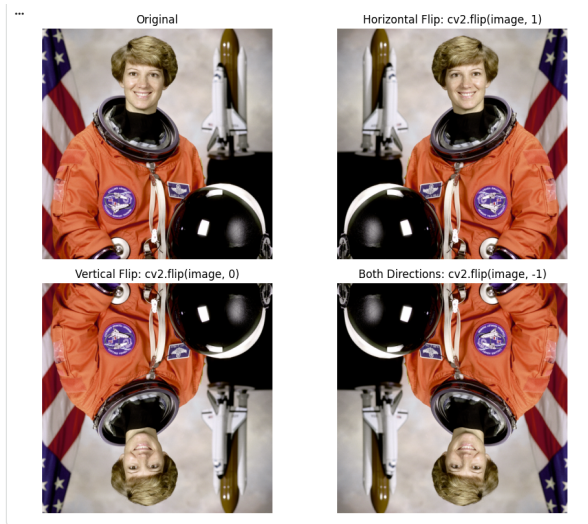
axes[1, 0].imshow(vertical_rgb)
axes[1, 0].set_title("Vertical Flip: cv2.flip(image, 0)")

axes[1, 1].imshow(both_rgb)
axes[1, 1].set_title("Both Directions: cv2.flip(image, -1)")

for ax in axes.ravel():
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Flip—Result



Saving an Image

Processed images can be written to disk using

```
cv2.imwrite()
```

The file format is determined by the filename extension.

- `image.jpg`
- `image.png`
- `image.bmp`

Example

```
cv2.imwrite("output.png", image)
```

Current Subsection

1 Python Image Libraries

- OpenCV
- Pillow
- scikit-image
- TensorFlow
- PyTorch

Pillow: Simple Image Manipulation

Pillow is a popular Python library for basic image handling.

- Open and save images
- Inspect image size and mode
- Convert between RGB, grayscale, and RGBA
- Resize, crop, rotate, and flip images
- Work with common formats such as PNG and JPEG

Import

```
from PIL import Image
```

Import Pillow and open an image

```
▶ from PIL import Image

image = Image.open("astronaut.png")

print(image)
print(type(image))

... <PIL.PngImagePlugin.PngImageFile image mode=RGB size=512x512 at 0x790BFB54B590>
    <class 'PIL.PngImagePlugin.PngImageFile'>
```

Inspecting a Pillow Image

Pillow provides useful properties for understanding an image:

- `image.size`
- `image.mode`
- `image.format`

Important

For Pillow,

```
image.size = (width, height)
```

Inspect size, mode, format

```
▶ print("Size:", image.size)  
  print("Mode:", image.mode)  
  print("Format:", image.format)
```

```
... Size: (512, 512)  
    Mode: RGB  
    Format: PNG
```

Displaying a Pillow Image

Pillow images can be displayed directly using

```
image.show()
```

or with Matplotlib.

Good News

Unlike OpenCV, Pillow uses the standard **RGB** channel order.

Display

```
▶ image.show()
```

```
▶ import matplotlib.pyplot as plt
```

```
plt.imshow(image)  
plt.axis("off")  
plt.show()
```

...



Image Modes in Pillow

Pillow uses the term **mode** to describe how pixel values are stored.

- RGB: red, green, blue
- L: grayscale
- RGBA: RGB + alpha transparency

Images can be converted using

```
image.convert()
```

Convert to grayscale

```
▶ gray = image.convert("L")  
  
print(gray.mode)  
  
plt.imshow(gray, cmap="gray")  
plt.axis("off")  
plt.show()
```

... L



In Pillow, "L" represents an 8-bit grayscale image with one intensity value per pixel; other common modes include "RGB" for color images, "RGBA" for color with transparency, and "1" for binary black-and-white images.

Convert to RGBA

```
▶ rgba = image.convert("RGBA")  
  
print(rgba.mode)  
  
plt.imshow(rgba)  
plt.axis("off")  
plt.show()
```

*** RGBA



In Pillow, "RGBA" represents a four-channel image containing Red, Green, Blue, and an Alpha channel that controls pixel transparency.

RGBA

If the original image is fully opaque, converting from RGB to RGBA does not visibly change it because the new alpha channel is set to full opacity, usually 255. To actually see the effect of alpha, we make it partially transparent:

```
• rgba = image.convert("RGBA")  
  
# Change the alpha channel  
rgba.putalpha(100)  
  
plt.imshow(rgba)  
plt.axis("off")  
plt.show()
```

...



If alpha is 255, the pixel is fully opaque, meaning its color is fully visible.

Resizing with Pillow

Pillow resizes images using

```
image.resize()
```

The size is specified as

```
(width, height)
```

Example

```
image.resize((224,224))
```

Resizing

```
▶ resized = image.resize((224, 224))

print("Original:", image.size)
print("Resized:", resized.size)

plt.imshow(resized)
plt.axis("off")
plt.show()
```

```
... Original: (512, 512)
    Resized: (224, 224)
```



Cropping with Pillow

Pillow crops an image using a rectangular box:

(left, top, right, bottom)

In other terms,

$(x_{\min}, y_{\min}, x_{\max}, y_{\max})$

where $x_{\min}$ is the left coordinate, $y_{\min}$ is the top coordinate, $x_{\max}$ is the right coordinate, and $y_{\max}$ is the bottom coordinate.

Cropping is performed with

`image.crop()`

Crop

```
▶ crop = image.crop((100, 100, 350, 350))
```

```
plt.imshow(crop)  
plt.axis("off")  
plt.show()
```

...



Rotating with Pillow

Images can be rotated using

```
image.rotate()
```

For example:

90°, 180°, 270°

Rotate

```
▶ rotated = image.rotate(45)  
  
plt.imshow(rotated)  
plt.axis("off")  
plt.show()
```

...



If you want to avoid cutting off corners:

```
▶ rotated = image.rotate(45, expand=True)  
  
plt.imshow(rotated)  
plt.axis("off")  
plt.show()
```

...



Flipping with Pillow

Pillow provides built-in transpose operations for flipping images.

- Horizontal flip
- Vertical flip

Flip

```
▶ horizontal = image.transpose(Image.Transpose.FLIP_LEFT_RIGHT)
  vertical = image.transpose(Image.Transpose.FLIP_TOP_BOTTOM)
```

```
▶ fig, axes = plt.subplots(1, 3, figsize=(12, 4))

  axes[0].imshow(image)
  axes[0].set_title("Original")

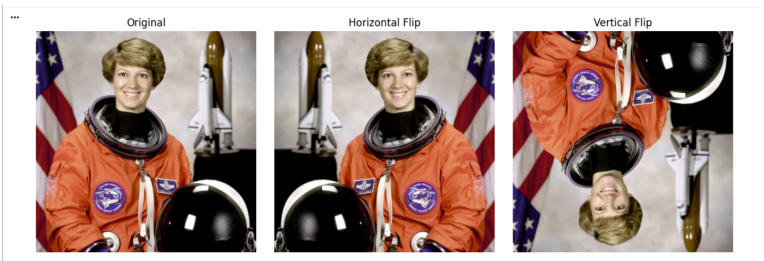
  axes[1].imshow(horizontal)
  axes[1].set_title("Horizontal Flip")

  axes[2].imshow(vertical)
  axes[2].set_title("Vertical Flip")

  for ax in axes:
      ax.axis("off")

  plt.tight_layout()
  plt.show()
```

Flip—Result



Saving Images with Pillow

Pillow saves images using

```
image.save()
```

The file format is usually inferred from the filename extension.

Examples

```
image.save("output.png")  
image.save("output.jpg")
```

Current Subsection

1 Python Image Libraries

- OpenCV
- Pillow
- **scikit-image**
- TensorFlow
- PyTorch

scikit-image: Image Processing for Teaching

scikit-image provides many useful image-processing tools.

- Built-in example images
- Filters and denoising
- Thresholding and segmentation
- Edge detection
- Image measurements

Import

```
from skimage import data, filters, color
```

Code

```
▶ from skimage import data
import matplotlib.pyplot as plt

image = data.astronaut()

plt.imshow(image)
plt.axis("off")
plt.show()

print(image.shape)
print(image.dtype)
```



(512, 512, 3)
uint8

Built-in Example Images

scikit-image includes several built-in images:

- `data.astronaut()` for RGB images
- `data.camera()` for grayscale images
- `data.coins()` for object detection examples
- `data.coffee()` for color images

Code

```
from skimage import data
import matplotlib.pyplot as plt

images = {
    "astronaut": data.astronaut(),
    "camera": data.camera(),
    "coins": data.coins(),
    "coffee": data.coffee(),
    "chelsea": data.chelsea(),
    "moon": data.moon(),
    "clock": data.clock(),
    "page": data.page(),
    "text": data.text(),
    "rocket": data.rocket(),
    "horse": data.horse(),
    "hubble": data.hubble_deep_field(),
    "immunohistochemistry": data.immunohistochemistry(),
}

fig, axes = plt.subplots(4, 4, figsize=(14, 14))
axes = axes.ravel()

for ax, (name, img) in zip(axes, images.items()):
    if img.ndim == 2:
        ax.imshow(img, cmap="gray")
    else:
        ax.imshow(img)

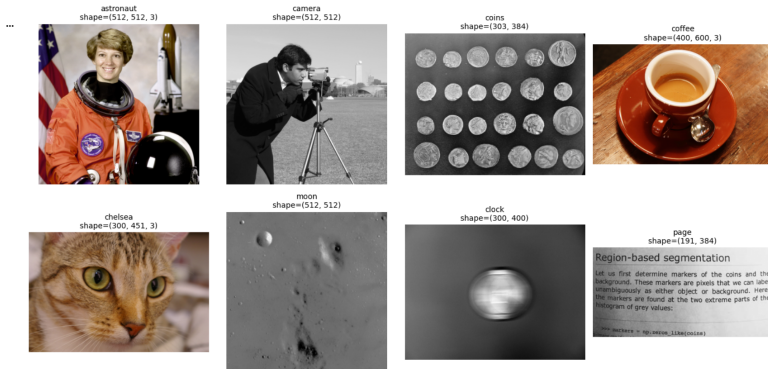
    ax.set_title(f"{name}\nshape={img.shape}", fontsize=10)
    ax.axis("off")

# Hide unused subplot cells
for ax in axes[len(images):]:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

A grayscale image usually has `ndim = 2`, which represents (height, width), while an RGB image usually has `ndim = 3`, which represents (height, width, channels).

Sample Images



Saving Astronaut Image

```
▶ from skimage import data  
  from PIL import Image  
  
  image = data.astronaut()  
  
  Image.fromarray(image).save("astronaut.png")
```

or:

```
▶ from skimage import data, io  
  
  image = data.astronaut()  
  
  io.imsave("astronaut.png", image)
```

Converting RGB to Grayscale

scikit-image can convert RGB images to grayscale using

```
color.rgb2gray()
```

The output values are usually floating-point numbers between

0.0 and 1.0

Code

```
▶ from skimage import color, data

image = data.astronaut()
gray = color.rgb2gray(image)

print(gray.shape)
print(gray.dtype)
print(gray.min(), gray.max())

plt.imshow(gray, cmap="gray")
plt.axis("off")
plt.show()
```

```
... (512, 512)
float64
0.0 1.0
```



Edge Detection

Edge detection highlights regions where pixel values change quickly.

A common edge detector in scikit-image is the Sobel filter:

```
filters.sobel()
```

Idea

Edges often correspond to object boundaries, texture changes, or strong brightness changes.

Classical edge detectors such as Sobel, Canny, and Prewitt are usually applied to grayscale images because they detect changes in pixel intensity. However, edge detection can also be applied separately to the red, green, and blue channels. For an RGB image, we can compute $E_R = \text{Sobel}(R)$, $E_G = \text{Sobel}(G)$, and $E_B = \text{Sobel}(B)$, and then combine them, for example using $E = \sqrt{E_R^2 + E_G^2 + E_B^2}$. This can detect boundaries caused mainly by color differences, even when the grayscale intensity difference is small.

Code

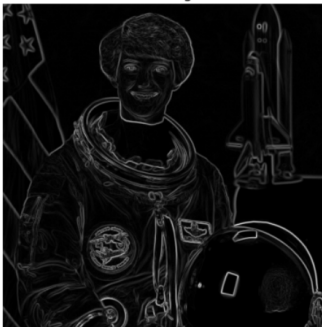
```
from skimage import filters

edges = filters.sobel(gray)

plt.imshow(edges, cmap="gray")
plt.axis("off")
plt.title("Sobel Edges")
plt.show()
```

...

Sobel Edges



Thresholding

Thresholding converts an image into a binary mask.

Pixels above a threshold become **True** or white.

Pixels below a threshold become **False** or black.

Example

```
mask = gray > threshold
```

Code

```
threshold = filters.threshold_otsu(gray)
mask = gray > threshold

print("Threshold:", threshold)

plt.imshow(mask, cmap="gray")
plt.axis("off")
plt.title("Binary Mask")
plt.show()
```

```
... Threshold: 0.388671875
```



Otsu's method tests possible threshold values and selects the threshold that maximizes the between-class variance, $\sigma_B^2 = w_0 w_1 (\mu_0 - \mu_1)^2$, where w_0 is the proportion of pixels in the dark group, w_1 is the proportion of pixels in the bright group, μ_0 is the mean intensity of the dark group, and μ_1 is the mean intensity of the bright group; therefore, Otsu chooses the threshold that makes σ_B^2 as large as possible, meaning the two groups are separated as clearly as possible.

Current Subsection

1 Python Image Libraries

- OpenCV
- Pillow
- scikit-image
- **TensorFlow**
- PyTorch

TensorFlow: Images for Deep Learning

TensorFlow represents images as tensors.

- Load image files
- Decode images into tensors
- Resize images for neural networks
- Normalize pixel values
- Build image datasets

Import

```
import tensorflow as tf
```

Loading an Image with TensorFlow: Method 1

A simple way to load an image is to use Keras utilities.

- `load_img()` loads the image as a PIL image.
- `img_to_array()` converts it to a numerical array.
- Good for quick experiments and individual images.

Workflow

Image File → PIL Image → NumPy-like Array

Method 1



```
import tensorflow as tf
```

```
img = tf.keras.utils.load_img("astronaut.png")
```

```
arr = tf.keras.utils.img_to_array(img)
```

```
print("Image type:", type(img))
```

```
print("Array type:", type(arr))
```

```
print("Shape:", arr.shape)
```

```
print("Dtype:", arr.dtype)
```

```
... Image type: <class 'PIL.PngImagePlugin.PngImageFile'>
```

```
Array type: <class 'numpy.ndarray'>
```

```
Shape: (512, 512, 3)
```

```
Dtype: float32
```

Displaying a TensorFlow/Keras Image

The image loaded with `load_img()` can be displayed directly.

The color channel order is normally

R, G, B

so no BGR-to-RGB conversion is required.

Compare with OpenCV

OpenCV loads color images as BGR by default, while TensorFlow/Keras typically works with RGB images.

Display

```
import matplotlib.pyplot as plt

plt.imshow(img)
plt.axis("off")
plt.show()
```

...



You can also display the array:

```
▶ plt.imshow(arr.astype("uint8"))  
plt.axis("off")  
plt.show()
```

...



Loading an Image with TensorFlow: Method 2

For TensorFlow pipelines, we can read and decode the image directly.

- `tf.io.read_file()` reads the file as raw bytes.
- `tf.image.decode_image()` converts the bytes into a tensor.
- `channels=3` forces a three-channel RGB image.
- Useful inside `tf.data.Dataset` pipelines.

Workflow

Image File → Raw Bytes → Tensor

Method 2: Loading an Image with TensorFlow

TensorFlow usually loads images in two steps:

```
tf.io.read_file()
```

reads the image file as bytes, then

```
tf.image.decode_image()
```

decodes it into a tensor.

Code

```
▶ import tensorflow as tf
import matplotlib.pyplot as plt

image_path = "astronaut.png"

image_bytes = tf.io.read_file(image_path)
image = tf.image.decode_image(image_bytes, channels=3)

print(type(image))
print(image.shape)
print(image.dtype)
```

```
... <class 'tensorflow.python.framework.ops.EagerTensor'>
(512, 512, 3)
<dtype: 'uint8'>
```

Displaying a TensorFlow Image

A TensorFlow image tensor can be displayed with Matplotlib.

For an RGB image, the tensor shape is usually

$$(H, W, 3)$$

Code

```
plt.imshow(image)
plt.axis("off")
plt.show()
```



Inspecting TensorFlow Images

Whenever an image is loaded, inspect its numerical representation.

Always check:

shape	dtype	value range
-------	-------	-------------

These checks help detect common image-pipeline problems.

Debugging Habit

Many bugs come from incorrect shape, data type, value range, or color-channel order.

Code

```
▶ print("Shape:", image.shape)
  print("Dtype:", image.dtype)

  print("Minimum:", tf.reduce_min(image).numpy())
  print("Maximum:", tf.reduce_max(image).numpy())
```

```
... Shape: (512, 512, 3)
    Dtype: <dtype: 'uint8'>
    Minimum: 0
    Maximum: 255
```

Resizing and Normalizing

Before training a neural network, images are often resized and normalized.

$$224 \times 224$$

is a common input size for vision models.

Normalization converts pixel values from

$$0-255 \rightarrow 0.0-1.0$$

Code

```
▶ image_resized = tf.image.resize(image, (224, 224))

image_normalized = image_resized / 255.0

print("Shape:", image_normalized.shape)
print("Dtype:", image_normalized.dtype)
print("Min:", tf.reduce_min(image_normalized).numpy())
print("Max:", tf.reduce_max(image_normalized).numpy())
```

```
... Shape: (224, 224, 3)
    Dtype: <dtype: 'float32'>
    Min: 0.0
    Max: 1.0
```

Standardization

Another common preprocessing technique is **standardization**.

$$x' = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

- Centers the data around 0.
- Values below the mean become negative.
- Values above the mean become positive.
- The resulting data typically has mean ≈ 0 and standard deviation ≈ 1 .

Important

Standardization is not always required. The preprocessing method should match the model and training pipeline.

Standardization

```
▶ img_float = tf.cast(image, tf.float32)

mean = tf.reduce_mean(img_float)
std = tf.math.reduce_std(img_float)

img_std = (img_float - mean) / (std + 1e-7)

print("Original mean:", mean.numpy())
print("Original std:", std.numpy())

print("Standardized mean:", tf.reduce_mean(img_std).numpy())
print("Standardized std:", tf.math.reduce_std(img_std).numpy())

print("Min:", tf.reduce_min(img_std).numpy())
print("Max:", tf.reduce_max(img_std).numpy())

... Original mean: 114.599
Original std: 81.23036
Standardized mean: 9.934107e-08
Standardized std: 0.9999999
Min: -1.4107902
Max: 1.72843
```

Displaying the Standardized Image

```
import matplotlib.pyplot as plt
```

```
plt.imshow(img_std)  
plt.axis("off")  
plt.show()
```

```
*** WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).
```



The problem is that the standardized image contains pixel values outside the valid Matplotlib RGB float range of $[0, 1]$, so `imshow()` clips them:

$$x_{\text{display}} = \min(1, \max(0, x_{\text{std}}))$$

The standardized image is still valid for CNN input because CNNs can process negative standardized values.

Adding a Batch Dimension

Neural networks usually expect a batch of images, not a single image.

A single RGB image has shape

$$(H, W, 3)$$

A batch of images has shape

$$(N, H, W, 3)$$

where N is the batch size.

Code

```
▶ batch = tf.expand_dims(image_normalized, axis=0)  
  
print(batch.shape)  
  
... (1, 224, 224, 3)
```

TensorFlow Image Pipeline

A basic TensorFlow image-preprocessing pipeline is:

file path → read bytes → decode image → resize → normalize → batch

Key Idea

TensorFlow prepares images in the same format expected by deep learning models.

Code

```
▶ def load_preprocess_image(path, size=(224, 224)):  
    image_bytes = tf.io.read_file(path)  
    image = tf.image.decode_image(image_bytes, channels=3)  
    image = tf.image.resize(image, size)  
    image = image / 255.0  
    return image  
  
image = load_preprocess_image("astronaut.png")  
  
print(image.shape)  
print(image.dtype)  
  
... (224, 224, 3)  
    <dtype: 'float32'>
```

OpenCV vs TensorFlow loading

Feature	OpenCV	TensorFlow
Object type	NumPy array	Tensor
Default color	BGR	RGB
Best for	Classical image processing	Model pipelines
Display issue	Convert BGR to RGB	Usually direct RGB
Pipeline use	Manual / NumPy	Strong <code>tf.data</code> support

Data Augmentation Preview

Data augmentation creates modified versions of training images without changing their class label.

Common augmentations include:

- horizontal flipping,
- small rotations,
- zooming,
- translation,
- brightness or contrast changes.

Why Use It?

Data augmentation increases the variety of training examples and can help reduce overfitting.

Important

Augmentation should usually be applied to the **training set only**.

Code

```
data_augmentation = tf.keras.Sequential([  
    tf.keras.layers.RandomFlip("horizontal"),  
    tf.keras.layers.RandomRotation(0.1),  
    tf.keras.layers.RandomZoom(0.1),  
])
```

```
▶ image_batch = tf.expand_dims(image_normalized, axis=0)  
  
    augmented = data_augmentation(image_batch, training=True)  
  
    plt.imshow(augmented[0])  
    plt.axis("off")  
    plt.show()
```

...



The value sets the maximum transformation range, while the exact rotation, zoom, or flip applied to each image is chosen randomly within that range.

Samples

```
fig, axes = plt.subplots(2, 4, figsize=(12, 6))

for ax in axes.ravel():
    augmented = data_augmentation(image_batch, training=True)
    ax.imshow(augmented[0])
    ax.axis("off")

plt.tight_layout()
plt.show()
```



Augmentation Intuition

A model should recognize the same object even when the image changes slightly.

Original $\rightarrow$ Flipped $\rightarrow$ Rotated $\rightarrow$ Zoomed $\rightarrow$ Brighter

Goal

Teach the model to focus on the important visual features of the object rather than memorizing one exact image.

Important

Only use transformations that preserve the correct label.

For example, some flips or rotations may change the meaning of digits, letters, medical images, or directional objects.

Code — include brightness too

```
▶ data_augmentation = tf.keras.Sequential([  
    tf.keras.layers.RandomFlip("horizontal"),  
    tf.keras.layers.RandomRotation(0.1),  
    tf.keras.layers.RandomZoom(0.1),  
    tf.keras.layers.RandomBrightness(0.2),  
])
```

Current Subsection

1 Python Image Libraries

- OpenCV
- Pillow
- scikit-image
- TensorFlow
- PyTorch

PyTorch: Images as Tensors

PyTorch is another popular deep learning library.

- Convert images to tensors
- Normalize images
- Build datasets and data loaders
- Train CNNs and vision transformers

Import

```
import torch
```

Code

```
▶ import torch
  from PIL import Image
  import matplotlib.pyplot as plt
  from torchvision import transforms

  image = Image.open("astronaut.png").convert("RGB")
  print(type(image))

... <class 'PIL.Image.Image'>
```

Converting an Image to a PyTorch Tensor

PyTorch models work with tensors.

A common conversion is:

PIL Image $\rightarrow$ PyTorch Tensor

Unlike NumPy and TensorFlow, PyTorch commonly uses:

(C, H, W)

instead of

(H, W, C)

Code

```
▶ transform = transforms.ToTensor()

tensor = transform(image)

print(tensor.shape)
print(tensor.dtype)
print(tensor.min())
print(tensor.max())

... torch.Size([3, 512, 512])
torch.float32
tensor(0.)
tensor(1.)
```

PyTorch Channel Order

For an RGB image, PyTorch usually stores dimensions as:

$$(C, H, W)$$

where

$$C = 3$$

for Red, Green, and Blue.

For example:

$$(3, 224, 224)$$

Code

```
▶ resized_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])

tensor = resized_transform(image)

print("Tensor shape:", tensor.shape)

... Tensor shape: torch.Size([3, 224, 224])
```

Displaying a PyTorch Tensor

Matplotlib expects images in the format:

$$(H, W, C)$$

but PyTorch usually stores images as:

$$(C, H, W)$$

So we need to rearrange the dimensions before displaying.

Code

```
image_for_display = tensor.permute(1, 2, 0)  
  
plt.imshow(image_for_display)  
plt.axis("off")  
plt.show()
```

...



`image_for_display = tensor.permute(1, 2, 0)` means:

- new axis 0 = old axis 1 = height
- new axis 1 = old axis 2 = width
- new axis 2 = old axis 0 = channels

Adding a Batch Dimension in PyTorch

A single image tensor has shape:

$$(C, H, W)$$

A batch of images has shape:

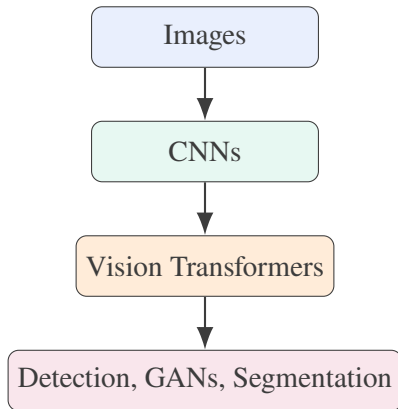
$$(N, C, H, W)$$

where N is the batch size.

Code

```
 batch = tensor.unsqueeze(0)  
  
print(batch.shape)  
  
... torch.Size([1, 3, 224, 224])
```

How this connects to the rest of the course



Congratulations!

Now you know about the most important image-processing and computer-vision libraries!

